CE 310

Python Tutorial — Bootstrap Confidence Intervals

Week 14 · Session 2 · Hands-on Walkthrough

University of Arizona · Dr. Wooyoung Jung · Fall 2026

Today's Plan — One Dataset, One Idea, Six Statistics

The TxDOT bid tabulation from Weeks 11 and 12, and a single idea applied six times: **resample the data, recompute the statistic, read the interval off the spread.**

By the end of this walkthrough you will know how to:

1. Build a bootstrap CI for a mean and set it against the analytical *t*-interval
2. Build a bootstrap CI for a **95th percentile** — a statistic with no formula
3. Run a **pairs bootstrap** for a regression slope using `rng.integers`
4. Bootstrap a coefficient and an R^2 together
5. Run a **permutation test** for a group difference
6. Bootstrap an exceedance probability, and compare interval widths

Libraries needed: `numpy`, `pandas`, `scipy`, `statsmodels`, `matplotlib`.

Step 1 — Setup, and Two Populations

⚠️ This week keeps **two** samples apart. `exc` is every bid on the item — winners and losers — and is what the price model is fitted to. `won` is the awarded rows only, and is what the agency actually paid. Bootstrap the wrong one and every number in Section A moves.

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
import statsmodels.formula.api as smf
from scipy import stats

N_BOOT = 1000;  SEED = 42;  THRESHOLD_USD = 100.0

df  = pd.read_csv('CE310_BidItems_2024_2026.csv')
exc = df[df['ItemDescription'] == 'EXCAV (ROADWAY)'].copy()
exc['LOW']  = (exc['LowBidder'] == 'Yes').astype(int)
exc['logq'] = np.log10(exc['Quantity'])
exc['logp'] = np.log10(exc['UnitPrice_USD'])

won   = exc[exc['LOW'] == 1].reset_index(drop=True)
price = won['UnitPrice_USD'].values
N, n  = len(exc), len(won)            # 4,369 bids | 555 awarded
rng   = np.random.default_rng(SEED)
```

Step 1 — One Generator, Consumed in Order

📌 `rng.integers(0, n, n)` draws **n row positions with replacement** — one resample. Pass it to `price[...]` or `exc.iloc[...]`.

> ⚠️ Every bootstrap in the in-class exercise draws from the **one** `rng` seeded here, in cell order. Re-run a cell out of order and the stream advances — every answer below it changes, and none of them will match the key. If you lose your place: **Runtime → Restart and run all**.

This is the single most common way to lose points in this in-class exercise. It costs nothing to avoid and it is not recoverable after the fact.

Step 2 — Bootstrap CI for the Mean Awarded Price

```
boot_means_P = np.array([
    price[rng.integers(0, n, n)].mean()
    for _ in range(N_BOOT)
])

ANSWER_A1 = round(boot_means_P.mean(), 3)
ANSWER_A2 = round(boot_means_P.std(), 5)
ANSWER_A3 = round(np.percentile(boot_means_P, 2.5), 3)
ANSWER_A4 = round(np.percentile(boot_means_P, 97.5), 3)

obs_mean = price.mean()
t_ci = stats.t.interval(0.95, df=n-1, loc=obs_mean, scale=stats.sem(price))
print(f"Bootstrap 95% CI : [{ANSWER_A3}, {ANSWER_A4}]")
print(f"Analytical t-CI  : [{t_ci[0]:.3f}, {t_ci[1]:.3f}]")
```

Step 2 — Expected Output & Interpretation

Expected: bootstrap **[25.551, 30.910]**, *t*-interval **[25.341, 30.936]**. Close, but not the same interval — and the difference is informative rather than noise.

	width	reach below the mean	reach above
Bootstrap	5.359	2.587	2.772
t-interval	5.595	2.797	2.797

> The *t*-interval is symmetric **because the formula makes it so** — it is the estimate plus and minus one quantity. Awarded prices are right-skewed (mean $28.14/CY, median $19.00/CY), and the bootstrap reaches further above the mean than below it. Nothing told it to; that asymmetry is in the data.

Step 3 — Bootstrap CI for the P95

```
boot_p95 = np.array([
    np.percentile(price[rng.integers(0, n, n)], 95)
    for _ in range(N_BOOT)
])

ANSWER_A5 = round(np.percentile(boot_p95, 2.5), 2)
ANSWER_A6 = round(np.percentile(boot_p95, 97.5), 2)

obs_p95 = np.percentile(price, 95)
print(f"Observed P95 : ${obs_p95:.2f}/CY")
print(f"Bootstrap 95% CI for P95 : [{ANSWER_A5}, {ANSWER_A6}] $/CY")
```

Expected: observed P95 **$94.30/CY**, CI **[66.50, 106.50]** — $40.00 wide, about **42% of the estimate**.

> No textbook gives a standard error for a sample percentile. Without resampling, an estimator quoting this not-to-exceed line has no interval to attach to it at all — not a narrow one, none.

Step 4 — Pairs Bootstrap for the Elasticity

```
boot_slopes = []
for _ in range(N_BOOT):
    idx = rng.integers(0, N, N)        # N here, not n — the model uses every bid
    exc_b = exc.iloc[idx]
    m_b = smf.ols('logp ~ logq', data=exc_b).fit()
    boot_slopes.append(m_b.params['logq'])
boot_slopes = np.array(boot_slopes)

ANSWER_A7 = round(np.percentile(boot_slopes, 2.5), 4)
ANSWER_A8 = round(np.percentile(boot_slopes, 97.5), 4)

m1 = smf.ols('logp ~ logq', data=exc).fit()
ols_ci = m1.conf_int().loc['logq']
```

📌 **Pairs** bootstrap: resample whole *rows*, so `logq` and `logp` stay attached. Refit inside the loop.

Step 4 — Results & Interpretation

Expected: bootstrap **[-0.1742, -0.1584]**, OLS **[-0.1735, -0.1596]**. The bootstrap interval is **13.7% wider**.

⚠️ This cell refits the model 1,000 times on 4,369 rows — a minute or so on Colab. A running cell is not a hung one. Do not interrupt it and re-run: that advances the `rng` and breaks every answer after this point.

Why wider? OLS assumes the residual spread is the same at every job size. It is not: residual standard deviation runs **0.314** on the smallest fifth of jobs against **0.210–0.235** on the rest. Small jobs price erratically, and the formula was not told.

Step 5 — Bootstrap Two Things at Once

```
boot_LOW, boot_R2 = [], []
for _ in range(N_BOOT):
    idx = rng.integers(0, N, N)
    exc_b = exc.iloc[idx]
    m_b = smf.ols('logp ~ logq + LOW', data=exc_b).fit()
    boot_LOW.append(m_b.params['LOW'])
    boot_R2.append(m_b.rsquared)
boot_LOW = np.array(boot_LOW)
boot_R2  = np.array(boot_R2)
```

One pass through the loop gives you an interval for **anything the fitted model exposes** — a coefficient, R^2, a predicted value, a ratio of two coefficients. The cost of the second statistic is zero.

Step 5 — Compute the CIs

```
ANSWER_B1 = round(np.percentile(boot_LOW, 2.5), 4)
ANSWER_B2 = round(np.percentile(boot_LOW, 97.5), 4)
ANSWER_B3 = round(np.mean(boot_R2), 4)
ANSWER_B4 = round(np.percentile(boot_R2, 2.5), 4)
```

Expected: `LOW` CI **[-0.0750, -0.0309]** — excludes zero, so the winner discount is real. R^2 mean **0.3402**, with a CI of **[0.3176, 0.3624]**.

> Two different lessons in one cell. The `LOW` interval comes out within 2% of the OLS one — here the formula holds up, and saying so is part of the answer. R^2 is the opposite case: `statsmodels` reports it as a bare number and offers no interval at all, so the bootstrap is not a check on the formula but the *only* source of one.

Step 6 — Permutation Test for the Winner Discount

```
low_yes  = exc.loc[exc['LOW'] == 1, 'logp'].values
low_no   = exc.loc[exc['LOW'] == 0, 'logp'].values
obs_diff = low_yes.mean() - low_no.mean()
combined = np.concatenate([low_yes, low_no])
n_yes    = len(low_yes)

perm_diffs = []
for _ in range(N_BOOT):
    shuffled = rng.permutation(combined)     # break the LOW label
    perm_diffs.append(shuffled[:n_yes].mean() - shuffled[n_yes:].mean())
perm_diffs = np.array(perm_diffs)

ANSWER_B5 = round(np.mean(np.abs(perm_diffs) >= np.abs(obs_diff)), 4)
```

Expected: `ANSWER_B5 = 0.0`, on an observed gap of **-0.0706**. The largest of the 1,000 shuffles reached only **0.0499**.

Step 6 — What "p = 0.0" Actually Says

> ⚠️ `ANSWER_B5 = 0.0` does **not** mean the probability is zero. It means that in 1,000 shuffles, not one produced a gap as large as the observed one — so the most the test can report is **p < 1/1000**. Write it that way. The parametric *t*-test, printed beside it, gives 1.65×10^{-7}.

To resolve it further, run more shuffles: 100,000 permutations resolve p down to 10^{-5}. Past that, the *t*-test's analytical tail is the practical instrument — the permutation test buys freedom from assumptions, not infinite resolution.

Step 7 — Exceedance Probability

```
boot_prob = np.array([
    (price[rng.integers(0, n, n)] > THRESHOLD_USD).mean()
    for _ in range(N_BOOT)
])
ANSWER_B6 = round(np.percentile(boot_prob, 2.5), 4)
ANSWER_B7 = round(np.percentile(boot_prob, 97.5), 4)
```

Expected: observed **0.0360** — 20 of the 555 awarded jobs cleared $100/CY — with a CI of **[0.0216, 0.0505]**.

> The upper end is **2.3×** the lower end. A point estimate of "about 3.6%" reads as precise and is not: on 20 events, the honest statement is that the rate is somewhere between one job in 46 and one in 20.

Step 7 — The Width Ratio

```
ols_ci_width  = ols_ci[1] - ols_ci[0]
boot_ci_width = ANSWER_A8 - ANSWER_A7
ANSWER_B8     = round(boot_ci_width / ols_ci_width, 3)

hc3 = m1.get_robustcov_results(cov_type='HC3')
print(f"OLS SE : {m1.bse['logq']:.5f} | HC3 SE : {hc3.bse[1]:.5f}")
```

Expected: `ANSWER_B8 = 1.137`, and HC3/OLS = **1.145**.

> Two corrections, arrived at independently: resampling knows nothing about the HC3 formula, and the HC3 formula does no resampling. They agree to within one part in a hundred. That agreement is what turns "the bootstrap gave a wider interval" from an anecdote into a finding.

Quick Check — Before You Start the In-Class Exercise

Verify your numbers, then open Wk14.03_In_Class_Exercise.ipynb

Confirm all four:

- **Mean awarded price CI** = [25.551, 30.910] — reaches further **above** the mean than below
- **P95 CI** = [66.50, 106.50] — $40.00 wide, and no formula could have produced it
- `LOW` **CI** = [-0.0750, -0.0309] — excludes zero
- **B8** = 1.137, against an HC3 ratio of 1.145

Question: B8 says the bootstrap interval for the elasticity is 13.7% wider than the OLS one, but the `LOW` coefficient's two intervals agree to within 2%. Same dataset, same model, same bootstrap. What is different about the two coefficients?

Quick Reference — Resampling

Task	Code pattern
Draw n indices with replacement	`rng.integers(0, n, n)`
Resample a plain array	`price[rng.integers(0, n, n)]`
Resample whole rows (pairs)	`exc.iloc[rng.integers(0, N, N)]`
Bootstrap 95% CI	`np.percentile(boot_array, [2.5, 97.5])`
Bootstrap standard error	`boot_array.std()`

Quick Reference — Comparison & Diagnostics

Task	Code pattern
Analytical *t*-CI	`stats.t.interval(0.95, df=n-1, loc=mean, scale=se)`
OLS coefficient CI	`m.conf_int().loc['logq']`
Permutation shuffle	`rng.permutation(combined)`
Exceedance probability	`(array > threshold).mean()`
Robust standard error	`m.get_robustcov_results(cov_type='HC3')`

Before You Submit

⚠️ Following this submission format exactly is essential — with a class this size, grading depends on it. Submissions that don't comply will be penalized.

1. **Run all cells first.** In Colab: Runtime → Run all. Wait until every cell shows a checkmark — no ⏳ spinners remaining.
2. **Download as .ipynb.** File → Download → Download .ipynb. Do not submit PDF or .py.
3. **Do not edit the `print` lines.** `print(f"ANSWER_A1 = {ANSWER_A1}")` must stay exactly as written, or that answer will not be counted.
4. **Do not change `SEED = 42`.** Results must use this exact seed to match the expected values.
5. **File name — `CE310_W14_<NetID>.ipynb`**, e.g., `CE310_W14_abc123.ipynb`.
6. **Upload** the `.ipynb` file to D2L Dropbox by **9:00 AM next Wednesday**.